



SaaS Browser Automation & Session Strategy

Distributed Worker Architecture & Anti-Detection Protocol

Executive Focus

This document presents the strategic and technical blueprint for integrating secure, long-lived user session persistence within our distributed browser automation infrastructure. By designing a robust "Login Once, Automate Anywhere" protocol, Pitch can safely scale headless operations across an elastic cloud node pool while evading modern automated detection systems.

Prepared For
Pitch Engineering & Product Teams

Author
Architecture & Strategy Group

Date
2026-05-22

Status
Technical Proposal

01 Executive Summary

The core engineering challenge for our SaaS platform is executing highly reliable browser automation tasks on behalf of our users without triggering anti-bot security systems (such as Cloudflare, Google reCAPTCHA, and Akamai).

Typical automated scraping setups get blocked instantly because they lack legitimate session history, use patched browser binaries, and lack persistent cookies or local storage. Our design overcomes these hurdles through an elegant three-tier architecture:

1. Login Once

Users authenticate once via a secure headed browser tunnel, solving CAPTCHAs and completing MFA/2FA manually.

2. Store Anywhere

The authenticated browser state is serialized as lightweight JSON/Tarball and stored securely in Postgres and S3.

3. Execute at Scale

Stateless cloud workers download the session on-demand, run real Chrome binaries, and lock profile states via Redis.

02 Authentication & Onboarding Flow

2.1 One-Time Headed Tunnel

To ensure the absolute highest login success rate, our SaaS avoids handling credentials directly whenever possible. Instead:

- We spin up a **headed** browser instance (visible display server) inside a sandboxed container.
- We proxy the browser's view directly into the user's dashboard web interface.
- The user logs in natively as they normally would, entering credentials, solving CAPTCHAs, and confirming authentication apps or SMS codes.
- Once the target site successfully redirects to the post-login dashboard, the system intercepts the event, captures the session state, and kills the container.

2.2 Session Capture Mechanisms

Depending on the target website's modern storage architecture, we employ two distinct extraction mechanisms:

Format	Description	Storage Medium
StorageState JSON	Cookies, sessionStorage, and localStorage extracted via Playwright's <code>storageState()</code> API. Perfect for 90% of classic SaaS apps.	PostgreSQL (JSONB)
Profile Tarball	Entire Chrome User Data Directory compressed as a <code>.tar.gz</code> archive. This preserves IndexedDB (used by Slack/Discord) and complex Service Workers.	Amazon S3 Bucket

03 Distributed Session Management

3.1 Concurrency and Distributed Locking

Because Chrome profile directories cannot be opened by multiple browser processes concurrently without database and preference corruption, we enforce a strict **single-worker execution lock**:

1. When a task is picked up from the queue, the worker requests a lock for `lock:user:id` via Redis.
2. The lock has a strict Time-to-Live (TTL) to prevent orphaned locks if the worker crashes.
3. Once the lock is acquired, the worker downloads the session JSON or extracts the tarball into its ephemeral local `/tmp` directory.
4. Google Chrome is executed, performs the automation task, and updates cookies or `localStorage`.
5. On successful exit, the worker serializes the updated state and writes it back to Postgres or S3.
6. The Redis lock is released, allowing subsequent tasks to execute.

3.2 Anti-Detection Strategy: Official Google Chrome

Standard Playwright Chromium binaries are trivially flagged by Cloudflare and Google Bot Management because they contain compiled-in debug protocols and lack proprietary media codecs.

We mitigate this risk by launching the **Official Google Chrome Stable** binary instead of Chromium:

✓ Genuine Fingerprints

Inherits legitimate user agent headers and TLS handshakes natively.

✓ Hardware Acceleration

Uses native GPU acceleration features to mimic real display output.

✓ Established Reputation

Uses existing browser certificates already trusted by safety providers.

✓ No Debug Flags

Completely disables automation metadata markers like `navigator.webdriver`.

04 Data Flow Lifecycle

1

Onboarding Trigger

User triggers connection. Control plane provisions an ephemeral headed browser worker connected to a secure display proxy.

2

Interactive Authentication

The user interacts with the remote browser in real-time, completing credentials, 2FA, SMS tokens, and solving CAPTCHAs.

3

Serialization & Secure Storage

Upon successful login detection, Playwright exports the session cookies and `localStorage` to S3 as an encrypted payload.

4

Asynchronous Task Queueing

Automated workflows are posted to Redis/BullMQ. High priority scheduler distributes tasks to available workers.

5

Lock Acquisition & Hydration

A distributed lock is secured in Redis. The worker pulls down the session state from S3 and restores the local directory.

6

Headless Execution & Commit

Real Chrome binary executes the task seamlessly in headless mode. Upon task completion, any updated cookies are pushed back to storage, and the lock is cleanly released.